

Demonstration of the GCC Compilation process

1. Original C program

```
root@1cd23b9c6365:/home/os# cat main.c
#include <stdio.h>

int main() {
    printf("Hello Aashish!\n");
    return 0;
}
root@1cd23b9c6365:/home/os#
```

Figure 1 : Original C source code

2. Preprocessing

```
root@1cd23b9c6365:/home/os# gcc -E main.c -o main.i
root@1cd23b9c6365:/home/os# head -30 main.i
# 0 "main.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "main.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
root@1cd23b9c6365:/home/os#
```

Figure 2 : Preprocessing of the C source file

The preprocessing stage processes directives such as **#include** and macros and produces the preprocessed source file **main.i**

3. Compilation

```
root@1cd23b9c6365:/home/os# gcc -S main.i -o main.s
root@1cd23b9c6365:/home/os# head -30 main.s
        .file     "main.c"
        .text
        .section   .rodata
.LC0:
        .string   "Hello Aashish!"
        .text
        .globl    main
        .type     main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq    %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq     %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq     .LC0(%rip), %rax
        movq     %rax, %rdi
        call     puts@PLT
        movl     $0, %eax
        popq     %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size     main, .-main
        .ident    "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
        .section   .note.GNU-stack,"",@progbits
```

Figure 3 : Compilation into assembly language

The compiler converts the preprocessed C code into assembly language, producing **main.s**.

4. Assembly

```
root@1cd23b9c6365:/home/os# gcc -c main.s -o main.o
root@1cd23b9c6365:/home/os# file main.o
```

```

root@1cd23b9c6365:/home/os# objdump -d main.o
main.o: file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0: f3 0f 1e fa          endbr64
 4: 55                   push    %rbp
 5: 48 89 e5             mov     %rsp,%rbp
 8: 48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
 f: 48 89 c7             mov     %rax,%rdi
12: e8 00 00 00 00      call    17 <main+0x17>
17: b8 00 00 00 00      mov     $0x0,%eax
1c: 5d                   pop     %rbp
1d: c3                   ret

```

Figure 4 : Assembly into object code

The assembler converts the assembly instructions into machine code and creates the object file main.o

5. Linking

```

root@1cd23b9c6365:/home/os# gcc main.o -o main
root@1cd23b9c6365:/home/os# file main
main: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically l
inked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=c45c6161da286e
7ccafffd71f15a084ee43ce546, for GNU/Linux 3.2.0, not stripped
root@1cd23b9c6365:/home/os#
root@1cd23b9c6365:/home/os#

```

Figure 5 : Linking to create the executable

The linker combines the object file with required libraries and other components to create the final executable main

6. Compilation

```

root@1cd23b9c6365:/home/os# ./main
Hello Aashish!
root@1cd23b9c6365:/home/os#
root@1cd23b9c6365:/home/os#

```

Figure 6 : Execution of the C program